

1 Refactor postmortem

18 September 2026 · Five architecture commits plus the current repair worktree

1.1 The answer, without the architectural sales pitch

We improved several important internals, but did not deliver the reduction in size or concentration of responsibility you reasonably expected. We bought bounded background work, stronger document identity, asynchronous saves and more testable service boundaries. We paid for them with more code, more lifecycle transitions, integration risk and your debugging time. The repeated reports of broken ordinary interactions mean that this is not an unqualified success.

The five architecture commits add **7,772 net physical Rust lines**. Including the uncommitted follow-up fixes, the increase is **8,689 lines, or 10.2%**. `app.rs` is still **13,980 lines**, down just **150 lines, or 1.1%** from the pre-refactor baseline. Counting its child modules, the application layer is **1,492 lines larger**, not smaller.

Approximately **4,216** of the added lines are identifiable test-file or inline test-module lines; the remaining **4,473** are a rough non-test remainder, not a compiler-derived production-only count. Tests explain a substantial share of the growth, but do not explain it away. The runtime complexity also increased.

Can we defend the growth? Partly. Protecting saves and bounding expensive work are worth extra implementation. Merely moving methods into another file, adding another representation of the same state, or passing architecture checks while native interactions fail is not an adequate return. The next phase should be stabilization followed by measured consolidation—not another broad rewrite.

1.2 Scope and evidence

Baseline: 27f644c (17 September, “fixes”), immediately before the first explicit architecture commit. This is also the baseline used by `docs/architecture-performance.md`. Endpoint: 3579b98 (“arch done”), plus the tracked Rust modifications present when this report was prepared.

This isolates the architecture campaign from earlier feature development such as tabs, miTeX, folding and configurable shortcuts. Those features already cost code at the baseline; charging all of them to this refactor would be misleading. Conversely, the repairs after “arch done” belong in the current cost picture.

Evidence has three different strengths:

- **Directly measured here:** Git history, source line counts, file organization, current state fields and actual before/after code.
- **Previously recorded evidence:** regression tests and the historical local profiling results in the architecture/performance documents. This review inspected those records; it did not repeat their benchmarks.
- **User-observed failures:** unreliable hover, keyboard navigation, link latency and Explorer sizing. These are real acceptance failures, but no commit bisect was performed here. Not every reported bug is proven to have been introduced by these five commits.

The current worktree also contains incomplete keyboard/link follow-up work. Neither its presence nor earlier passing Rust checks establishes that the reported native failures are fixed. The accidental launch of a September 8 app bundle was not valid evidence about the current build.

2 What actually grew?

2.1 Exact physical Rust line counts

Count each tracked `.rs` file once, including blanks, comments, tests, build support, core and `xtask`. Exclude dependencies, target, lockfiles, generated PDFs, screenshots, Markdown/Typst reports and profiling logs. These are **physical source lines**, not executable statements, binary size or memory consumption.

Snapshot	Rust files	Rust lines	Step delta	app.rs
27f644c baseline	111	85,253	—	14,130
45f1116 architecture change	116	85,734	+481	14,130
53d315a arch	123	87,456	+1,722	13,844
bb693d7 idk arch	129	88,339	+883	13,589
a17e6c9 more arch	132	89,244	+905	13,619
3579b98 arch done	138	93,025	+3,781	13,847
Current worktree	138	93,942	+917	13,980

The committed Rust diff is **13,172 added / 5,400 removed**. That is 18,572 lines of diff churn for 7,772 lines of net growth. The uncommitted Rust diff adds 1,192 and removes 275. Churn is not another measure of net size: relocation can count the same implementation as both a deletion and an insertion.

The first architecture commit’s whole-repository diff reports about 30,000 insertions, mostly because it includes audit documents, PDFs and retained measurement evidence. Its Rust net increase is only 481. Using that whole-repo figure as “code bloat” would be wrong.

2.2 Tests versus the rest

Category	Baseline	Current	Net
Test-only file convention	10,297	11,703	+1,406
Inline test-module suffix estimate	18,081	20,891	+2,810
Remaining Rust estimate	56,875	61,348	+4,473
Exact total	85,253	93,942	+8,689

Test-only files are paths under `tests/`, files named `tests.rs`, `test_allocations.rs`, or ending `_tests.rs`. The inline estimate counts the suffix starting with a top-level `#[cfg(test)] / mod tests {` in other files. It is deliberately labeled an estimate: test helpers elsewhere, unusual module placement, benchmarks and QA/profiling code make this different from an exact production build’s source footprint. The remaining bucket still includes comments, blanks, `xtask/build` tooling and some test-only helpers.

On that transparent approximation, about **48.5% of the net growth is test code** and **51.5% is the remainder**. “It is all tests” is false. “All 8,689 lines run in the UI” is also false.

2.3 Where the implementation cost went

The following groups count complete files, including their tests; groups are selected explanatory slices, not an additive decomposition of the repository.

Files grouped	Before	Current	Net
Compiler, PDF service/pages/residency, preview	2,542	4,034	+1,492
Project index, index runner, workspace/service	1,307	2,681	+1,374
Tinymist, transport/protocol, synchronization	4,423	5,298	+875
Git module and all descendants	4,030	4,764	+734
app.rs plus all app descendants	32,745	34,237	+1,492

The three new save files total another 1,062 lines, but that is **not** a net save subsystem increase: some save code was removed from `app.rs` and `tabs.rs`. Likewise, `compiler.rs` shrinking by 574 lines does not mean the PDF subsystem shrank. Its extraction and new page-management responsibilities made the group substantially larger. This distinction should appear in every future refactor PR.

3 What we gained—and what each gain cost

3.1 Document identity is substantially less fragile

Before, the active document lived outside the parked-tab store; parallel vectors and positional indices had to remain aligned. After, records stay keyed by stable IDs and display order is separate. This directly reduces the

number of things a reorder or tab switch must mutate together. It is a correctness gain, not an automatic performance win: keyed lookup and widget-state restoration still cost work, and the model retains an empty presentation record.

Before — selected fields, 27f644c:src/app/tabs.rs:

```
pub(super) struct Tabs {
    pub(super) parked: Vec<Option<ParkedTab>>,
    pub(super) active: usize,
    pub(super) preview: usize,
    // ...
    ids: Vec<u64>,
    // ...
}
```

After — selected fields, current src/app/tabs.rs:

```
pub(super) struct Tabs {
    records: BTreeMap<u64, TabRecord>,
    order: Vec<u64>,
    active: Option<u64>,
    preview: Option<u64>,
    // ...
    empty_record: TabRecord,
    // ...
}
```

The benefit is the eliminated “active hole” and positional remapping. The remaining cost is that many tab operations are still impl EditorApp methods, and save/preview/service transitions still cross the app-wide object. Stable IDs are worth keeping; they do not by themselves finish the ownership refactor.

3.2 Save IO moved off the UI path, with stronger completion identity

Before — contiguous excerpt, 27f644c:src/app.rs, save path:

```
let saved_fingerprint = fingerprint(save.source().as_bytes());
match atomic_write(save.path(), save.source().as_bytes()) {
    Ok(durability) => {
```

That write runs in the calling app method. The baseline already had prepared saves, atomic writing and some conflict/close guards; the refactor did **not** invent safe saving from nothing.

After — contiguous excerpt, current src/app/saves.rs:

```
let input = SaveInput::new(request, expected, intent, continuation);
if let Err(error) = self
    .save_job
    .start_and_repaint("Save document", context, move || {
        Ok(crate::save_io::execute(input))
    })
```

Current src/save_io.rs performs disk-state revalidation inside AtomicFileWriter::write_checked. Owner/tab/document keys and durable receipts decide whether a completion can clear dirty state or continue a close. Active and parked saves use the same path. This is defensible added complexity because slow disk work no longer belongs on UI dispatch and stale completions must not approve newer edits.

But asynchrony introduces another interval in which the tab can move, close or change path. The follow-up diff adds previous_sync_path and substantial parked Save As rebinding logic. That is evidence that the original extraction missed an ownership transition—not evidence that “background save” alone solved the problem. The app-owned lease also does not make our writes atomic against arbitrary external programs. Preserve that limitation in claims and tests.

3.3 Index work is bounded instead of merely ignoring stale answers

Before — current-job replacement in 27f644c:src/worker.rs:

```
self.receiver = None;
let (sender, receiver) = mpsc::channel();
let spawned = thread::Builder::new().name(name.into()).spawn(move || {
    let _span = crate::performance::span("worker.job");
    let result = work();
});
```

Dropping a receiver prevents use of an old result; it does not stop the old job. The baseline index caller supplied a complete `analyze_project` closure to that mechanism. Rapid requests could therefore continue obsolete work concurrently.

After — actual limits and worker creation in src/index_jobs.rs (two excerpts):

```
const CONCURRENCY: usize = 2;
const MAX_PENDING_BYTES: usize = 64 * 1024 * 1024;
const MAX_REQUEST_BYTES: usize = 32 * 1024 * 1024;

for index in 0..CONCURRENCY {
    let queue = queue.clone();
    thread::Builder::new()
        .name(format!("tiptotyp-index-read-{}", index))
        .spawn(move || worker_loop(&queue))
        .expect("project-index worker must start");
}
```

The specialized runner replaces pending work, supports cancellation and routes results to the owner. This is a genuine bounded-work improvement. It also adds a scheduler, queue accounting and admission semantics. Follow-up tests for rejected replacements preserving queued/active work show why those semantics needed more scrutiny. Keep it specialized; a generic executor framework would increase cost without establishing another benefit.

3.4 PDF residency now scales with demand, not all decoded pages

Before — selected lines from the decode loop in 27f644c:src/compiler.rs (omissions marked):

```
let mut pages = Vec::with_capacity(page_paths.len());
for (index, page_path) in page_paths.into_iter().enumerate() {
    // ... read and decode the page ...
    pages.push(PreviewPage {
        size: [width as usize, height as usize],
        rgba: decoded.into_raw(),
        // ... links ...
    });
}
```

After — selected definitions from src/pdf_pages.rs and src/pdf_residency.rs:

```
const MAX_PAGES_PER_REQUEST: usize = 12;

pub(crate) struct RasterPageKey {
    pub(crate) artifact: ArtifactKey,
    pub(crate) page: usize,
    pub(crate) dpi: u32,
    pub(crate) appearance_revision: u64,
}

pub(crate) const DECODED_PIXEL_BUDGET: usize = 96 * 1024 * 1024;
pub(crate) const TEXTURE_BUDGET: usize = 192 * 1024 * 1024;
```

Page catalogs, bounded visible-range requests and separate residency leases replace all-page decoded ownership. Artifact/appearance keys protect against late results. This is one of the strongest reasons to accept more code: the resource problem cannot be solved by moving the old loop into a smaller file.

These budgets are **accounting policies**, not a hard ceiling on total process or GPU memory. Evicted resources are dropped by their owning UI thread, transient allocations exist, WebKit has separate resources, and an oversized page can be admitted alone. Do not present 96/192 MiB as an OS-enforced bound.

3.5 Other worthwhile improvements

- Shared workspace observation uses one subscription service per canonical root rather than redundant per-window scans, while preserving local tree state.
- Tinymist transport/protocol code is separated from UI; synchronization has typed, versioned effects and stale-reply admission checks.
- Git execution/codecs are separated from rendering; repository mutation leases and checked patches protect unrelated staged work.
- Completion edits have an atomic, version-checked transaction boundary, and the completion view has explicit inputs/actions.
- Tool resolution/capabilities, child-view lifetime and process cleanup have better named contracts and focused tests.

None of these imply that every native interaction is now reliable. A good local boundary can still be connected incorrectly by the application orchestrator.

4 What did we lose?

4.1 Simplicity, integration confidence and reviewability

The old implementation was too concentrated, but it often executed a sequence in one place. We now have more requests, effects, tokens, leases, admission checks and result-routing steps. That complexity is sometimes necessary; it is not free. Many of the new contracts only pay off if **every adapter** follows them.

The final architecture commit alone changes 38 files and adds 3,781 net Rust lines. It combines tab ownership, synchronization, preview transitions, bounded PDF work, indexing, shared observation and native lifecycle work. That is too many behavioral seams for a single acceptance checkpoint. Commit subjects such as “idk arch” and “arch done” give little help identifying the invariant or regression risk when debugging later.

The result is a gap between “locally tested components” and “application works when you click around.” That gap has cost user time. The 107-item manual checklist was a poor response: it shifted verification work onto the person already finding the bugs. Its reduced replacement is a smoke test; deeper reproduction, automated coverage and fault injection remain developer work.

4.2 Native behavior was not sufficiently established

Observed issue	Why existing evidence was insufficient	What acceptance needs
Hover handoff/dismissal	Triangle math can pass while another timer or owner-lifecycle path closes the card.	Actual source → native card → scrolling → exit interaction, with owner-lifetime tests.
Option/Cmd navigation	A synthetic egui key proves editing only after delivery, not AppKit responder routing.	Native preview-to-source focus transfer followed by real word/line navigation.
Slow repeated link clicks	Rendering tests do not establish launch latency, dismissal or single activation.	Injected launcher tests for admission/reaping plus a current-build native activation.
Tiny Explorer	A default/minimum test does not cover persisted widths and constrained layout writeback.	Startup, narrow/wide resize and hide/reopen as one persistence sequence.
Index jargon	A technically correct count is not an actionable user explanation.	Name the file/expression, explain the limit, distinguish it from compile failure.

The uncommitted fixes directly show additional missed seams: rejected index admission, parked Save As identity, stale workspace events, PDF cancellation and residency, and hover syntax reuse. Some correct newly introduced behavior; some expose older weaknesses. Without a bisect, it would be dishonest to assign every one to a particular refactor commit.

Todo items 225/226 being checked despite later reports that the fixes do not work is an acceptance-process failure. Completion must mean the requested behavior is demonstrated at the relevant layer—not merely that a plausible patch and unit tests exist. This report does not silently change those todo statuses.

4.3 Performance claims must stay narrower than the changes

Historical evidence in docs/architecture-followup.md records:

- An index-runner synthetic comparison: 100 requests used 100 old threads versus two shared workers; recorded completion time fell from about 193 ms to 11 ms. This establishes that workload's benefit, not general typing latency.
- A synthetic PDF accounting probe: for 100 pages at 8 MiB per page resource, the modeled all-page total was 1,600 MiB versus 24 MiB resident after the new sequence. This is not an actual OS/GPU memory measurement.
- A native setter trace model: 300 calls over 100 unchanged frames became zero after initial creation. That is eliminated redundant work, not a measured end-to-end frame-rate improvement.

The earlier docs/architecture-performance.md comparisons used Apple M2 Max, arm64 macOS 14.6.1 and matched optimized builds. They found no material regression in selected steady-state workloads and caught an indexing cost regression that was repaired. But those measurements precede much of the final campaign and exclude several active-interaction, child-process and GPU costs. They cannot certify the current dirty worktree or negate the user's stalls.

The UI/native views still require main-thread coordination. Moving expensive work off that thread is useful; assigning every window an arbitrary UI thread is not the natural next step and can violate native toolkit requirements.

5 Why is app.rs still so large?

5.1 We extracted services, not the whole application coordinator

The exact line trajectory is revealing: 14,130 → 13,844 → 13,589 → 13,619 → 13,847 → 13,980. Early removals were mostly spent again on adapter glue and new behavior. The best committed reduction was 541 lines; the current reduction is only 150. The full app family grew from 32,745 to 34,237 lines.

Excluding conventionally test-only child files, the family still grew from 24,751 to 25,657 physical lines. There are some test helpers in that remainder, but this is enough to disprove the idea that only the app tests got larger.

The EditorApp struct had 128 named field declarations before; it has 129 now. That count includes conditional-platform fields and is not the simultaneous runtime field count on one target. Roughly 255 indented fn declarations remain in app.rs, versus 254 before; this is a textual indicator, not an AST count of only EditorApp methods. Both indicators show essentially unchanged central coordination pressure.

5.2 Moving a method did not narrow what it can touch

The save file provides the clearest example. It improves IO behavior, but its application boundary still starts like this:

```
// src/app/saves.rs
use super::*;

impl EditorApp {
    // ... save admission and completion routing ...
}
```

The editor, native-view, settings and tab files also contain methods on the same large object. Rust module privacy is not a strong isolation boundary here: child modules can access their parent's private state. This is file organization, not independent ownership. The new save_io service is a real effect boundary; the extracted impl EditorApp by itself is not.

The refactor left the app responsible for command routing, dialogs, document lifecycles, preview/service orchestration, source navigation, hover/completion, Explorer/index rendering, Settings propagation, Git integration and status. It also kept thousands of standalone helpers in the root file.

5.3 Concrete hotspots still in the root file

Function	Lines	Responsibility still centralized
show_workspace	331	Layout, Explorer interaction and pane composition
handle_shortcuts	296	Focused-owner routing and command handling
ui_in_window	288	Per-frame sequencing across subsystems
new_session	284	Initialization of app-wide state/services
receive_tinymist_events	250	Protocol results applied to app state
show_toolbar	197	Menus, tabs and toolbar actions
restart_tinymist_with_handoff	186	Service/preview transition coordination

These are physical function spans in the inspected worktree, including comments and blanks. Approximately the final 5,160 lines—from `request_pdf_pages` onward— are predominantly free helpers and supporting types: PDF texture preparation, Explorer rendering, menu/popup geometry, text transforms, package browsing, shortcut editing and filesystem helpers. They are not an enormous inline unit test module: the main tests already live in `src/app/tests.rs` (7,171 lines).

Therefore “**move the tests out**” is **not the answer**. Nor is moving the final 5,160 lines wholesale into `app_helpers.rs`: that would shrink a filename while preserving the same dependencies and making related code harder to find.

6 Can we defend the bloat?

Use three different verdicts rather than a single yes/no:

Defensible and worth keeping: stable document IDs; protected background saves; bounded index work; visible-page PDF residency; stale-result checks at mutation boundaries; shared workspace observation. These have concrete failure modes or resource bounds to justify their machinery. Removing checks merely to hit a line target would be a regression.

Useful but only partly paid off: protocol/view extractions, typed preview effects and new lifecycle models. They make unit testing easier, but their value is diminished while `EditorApp` remains the integration bottleneck and real interactions fail. They need smaller, clearer adapters and fewer state owners, not another layer around the same mutable object.

Not yet justified as a completed outcome: the claim that the architecture work has simplified the whole app or established broad performance/reliability. The measured root-file reduction is negligible, roughly 4.5k extra non-test remainder lines remain, and acceptance failures are unresolved. We should retain the good machinery but stop calling this campaign “done.”

There is no demonstrated reason to delete 8,689 lines indiscriminately or revert the entire campaign. A wholesale revert would throw away real safety/resource improvements and mixes in further risk. Conversely, “more tests” is not a blank cheque for permanently growing the production state space.

7 Next steps: fix the outcome, not the metric

7.1 First: close the known acceptance gap

Freeze feature work and broad ownership changes while resolving hover handoff, native keyboard delivery, link activation and persisted Explorer width. Use the exact freshly built executable, recording path/hash; never validate by app-name lookup. Reproduce before patching and trace the input/result across the relevant boundary. A regression test at the wrong layer is insufficient.

Add targeted developer-owned tests: injected link launching; lifecycle sequences for source/card/owner closure; constrained-width persistence; actual native focus handoff where the environment supports it. If native verification cannot be performed, explicitly leave that outcome unverified. Do not ask the user to run a hundred checks to compensate. Preserve the short `checklist.typ` as an optional smoke test, not the primary test harness.

7.2 Then: small extraction with an explicit deletion plan

These are proposed next changes, not implemented work or guaranteed line savings. Each should be a separately reviewable commit with before/after accounting.

Order	Scoped change	Boundary / deletion	Exit condition
1	Move cohesive leaf UI/helpers	Move Explorer render helpers to the Explorer view, popup/menu geometry to their views, package browser to its own module. Explicit imports; delete old copies.	Root-file reduction; essentially neutral total production lines; unchanged behavior. No generic helper dumping ground or new state model.
2	One navigation/focus handoff	After reproducing the bug, centralize document/range navigation and native/editor focus transfer used by Problems, index and preview.	Callers use one tested path; remove duplicated focus sequencing. Word navigation works after every entry point, including secondary windows.
3	Narrow Explorer ownership	Give the view explicit tree/index/selection inputs and typed actions. Keep watcher effects outside painting; relocate corresponding state out of EditorApp.	No full EditorApp parameter/import in the view; fewer actual app-owned decisions, not just a new struct wrapping unchanged access.
4	Thin save completion integration	Consolidate active/parked identity updates and continuation handling around the existing save receipt model; preserve disk lease/durability checks.	Remove duplicated branches only with same-path, Save As, close and stale-result tests. Total non-test implementation should be net-negative.
5	Thin preview/synchronization adapters	Inventory where connection, content, native visibility and generation are written; eliminate duplicate policy using the existing controllers.	One decision owner per policy. Smaller receive/restart paths, no new executor, no second retry counter, no per-frame source-copy or repaint regression.

Do not run all five tracks in parallel. Leaf relocation is low risk; changing owner identity or asynchronous sequencing is not. Keep native objects on their required thread. Broad replacement of the UI toolkit or a new generic event bus is outside this plan and currently unsupported by evidence.

7.3 Measure success on two axes

Axis one: root-file size and dependency concentration. A first milestone of app.rs below roughly 10,000 lines is plausible by relocating cohesive existing helpers/views; it is a proposed readability target, not a proven saving or an architectural victory. Review who can mutate each state, not just the number. A later sub-5,000-line composition root requires genuine ownership changes and must not be promised as a mechanical split.

Axis two: total implementation and state complexity. Moving code must not be advertised as reducing total code. For consolidation commits, require a net reduction in non-test implementation and name the deleted branch/state/adaptor. Allow growth only for a specified correctness requirement or measured resource benefit, with the cost visible before acceptance. Keep tests separately counted; do not cut valuable regression coverage to manufacture a smaller diff.

Record for each commit: root-file lines, module-family lines, test/non-test accounting, state/ownership change, deleted implementation, targeted test evidence and relevant performance workload. Avoid hard timing thresholds in ordinary CI; use deterministic bounds plus matched optimized measurements when performance changes. Source-string architecture guards help catch obvious dependencies, but must not replace semantic or native integration tests.

Use commit subjects that name an invariant, such as “Preserve parked Save As identity on delayed completion,” and split behavior fixes from mechanical moves. This makes review, rollback and later bisection practical. The next report should be able to point to **deleted complexity**, not just newly named modules.

8 Reproduction and source ledger

The counts are snapshots, not live values. This report itself and the checklist are not Rust and therefore do not change the reported total. No application code was changed for this postmortem.

Exact history and diff commands:

```
git log --reverse --oneline 27f644c..3579b98
git diff --numstat 27f644c 3579b98 -- '*.rs'
git diff --numstat 3579b98 -- '*.rs'
git show 27f644c:src/app.rs | wc -l
wc -l src/app.rs
```

Reproduce total physical lines without checking out or disturbing the worktree:

```
import pathlib, subprocess
def git(*args):
    return subprocess.check_output(["git", *args])
for ref in ["27f644c", "3579b98", "WORKTREE"]:
    paths = (git("ls-files", "-z") if ref == "WORKTREE" else
             git("ls-tree", "-rz", "--name-only", ref))
    paths = [p for p in paths.decode().split("\0") if p.endswith(".rs")]
    total = 0
    for p in paths:
        data = (pathlib.Path(p).read_bytes() if ref == "WORKTREE"
               else git("show", ref + ":" + p))
        total += len(data.splitlines())
    print(ref, len(paths), total)
```

Primary local sources:

- docs/architecture-performance.md: baseline, historical matched measurements and limitations; not final-campaign acceptance.
- docs/architecture-followup.md: bounded PDF/index work, shared observation, tab ownership, synchronization and their recorded probes.
- docs/background-saves.md, src/save_io.rs, src/app/saves.rs: disk effects, receipt routing and the follow-up parked-save changes.
- src/app/tabs.rs, src/tinymist_sync.rs, src/index_jobs.rs, src/pdf_pages.rs, src/pdf_residency.rs, src/workspace_service.rs: current ownership/resource implementations.
- src/app.rs, src/app/tooltips.rs, src/app/editor_view.rs, src/child_view.rs: remaining integration and native interaction seams.
- tests/architecture_boundaries.rs: useful structural guardrails, explicitly not substitutes for behavioral adapter tests.
- todo.typ 184–227 and the user's follow-up reports: intended scope versus outstanding acceptance issues. Checked status is not independent evidence.

Bottom line: preserve the concrete safety and bounded-resource gains. Admit that code size and centralization barely improved. Fix the known interactions, then make each small consolidation delete a named responsibility from the central app and, where feasible, delete implementation overall. That is the missing second half of this refactor.